

Some Tools and Deep Learning Scaling

Paul Rodriguez, PhD
(SDSC)

June 2026

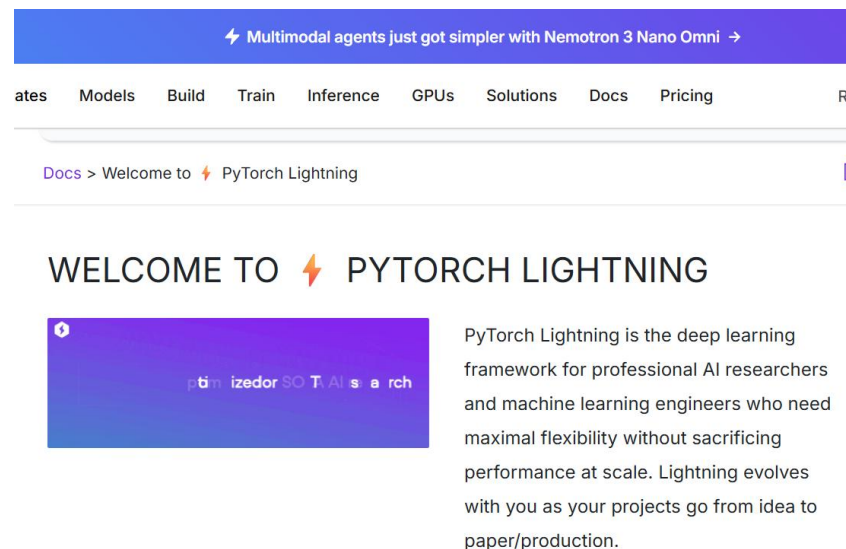
Outline

- The Pytorch Lightning tool for higher level programming and multimode/multidevice training
- The Raytune tool for hyperparameter searching and multimode/multidevice execution
- Deepspeed for saving memory by sharding parameters across devices

These are all packages that ‘wrap’ around your code to perform tasks easier

Pytorch Lightning framework

- <https://lightning.ai/docs/pytorch/stable/>
- Several steps to wrap your code in a “trainer” function
- Pytorch Lightning will run the train loop, set up DDP, do early stopping,



Pytorch Lightning framework

- The “trainer” function

‘strategy=DDP’ tells Lightning to initialize DDP and wrap model with DDP

‘plugins=[mpi_env] tells Lightning to use mpi to get rank, world size information

(other arguments are as usual)

```
import lightning as L
```

```
trainer = L.Trainer(
    accelerator = “gpu” or “cpu”
    max_epochs=epochs,
    devices      =numdevices_2use,
    num_nodes    =numdevices_2use,
    strategy     =“ddp”,
    plugins      =[mpi_env],
    log_every_n_steps=5,
    callbacks=[timer]  # some timing info
)
```

Pytorch Lightning framework

- The fit function

Pass the model and data loaders

```
import lightning as L
```

```
trainer = L.Trainer(  
    accelerator = "gpu" or "cpu"  
    max_epochs=epochs,  
    devices      =numdevices_2use,  
    num_nodes    =numdevices_2use,  
    strategy      ="ddp",  
    plugins       =[mpi_env],  
    log_every_n_steps=5,  
    callbacks=[timer] # some timing info  
)
```

```
trainer.fit(model, train_loader, val_loader)
```

Pytorch Lightning framework

- So “fit” knows epochs and `model.forward`,
- It run the backward prop function
- But for flexibility you code the train/validation steps, optimizer
- Pytorch Lightning provides some other hooks for flexibility

```
def training_step(model, batch, batch_idx):  
    .... (same code as before)  
    return loss  
  
def validation_step(model, batch, batch_idx):  
    .... (same code as before)  
    return  
  
def configure_optimizers(self):  
    optimizer=torch.optim.Adam( ....  
    return optimizer  
  
def on_train_start(self): # a hook
```


Pytorch Lightning framework

Pytorch Lightning will also help:

- Perform 'cuda' calls and move items to gpu device as needed
- Log training/validation loss

```
def training_step(model, batch, batch_idx):  
    .... (same code as before)  
    return loss
```

```
def validation_step(model, batch, batch_idx):  
    .... (same code as before)  
    return
```

```
def configure_optimizers(self):  
    optimizer=torch.optim.Adam( ....  
    return optimizer
```

```
def on_train_start(self): # a hook
```

Pytorch Lightning demo/tutorial

- Login to expanse, go to ciml26 repo, 4.3 Tools
- Review then run: *sbatch run-pyt-main-cpu1-pyt.sb*
- Look for stdout.. File
- Optionally change number of nodes and/or tasks, adjust the 'mpirun' command parameters

Pytorch Lightning demo/tutorial

- To view execution:
`queue -u $USER`
`ssh exp-#-##`
`top -u $USER`

```
p4rodrig@exp-2-26:~/WKSHOPS/CIML26/ToolswScaling
top - 18:47:22 up 23 days, 2:43, 1 user, load average: 15.86, 9.47, 4.98
Tasks: 1858 total, 1 running, 1826 sleeping, 0 stopped, 31 zombie
%Cpu(s): 8.2 us, 38.5 sy, 0.0 ni, 52.9 id, 0.0 wa, 0.4 hi, 0.0 si, 0.0 st
MiB Mem : 257485.8 total, 236214.6 free, 13522.4 used, 7748.8 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 241841.0 avail Mem

 147300 p4rodrig 20 0 13.3g 980736 254392 R 184.8 0.4 0:11.63 python3 70
 147255 p4rodrig 20 0 13.4g 993364 257136 S 803.3 0.4 0:36.89 python3 21
 147253 p4rodrig 20 0 13.4g 993068 257128 S 757.6 0.4 0:33.69 python3 52
 147300 p4rodrig 20 0 13.4g 991924 257180 S 755.6 0.4 0:34.60 python3 69
 147301 p4rodrig 20 0 13.4g 993840 257140 S 739.5 0.4 0:33.91 python3 93
 147303 p4rodrig 20 0 13.4g 990784 257108 S 733.2 0.4 0:33.65 python3 123
 147302 p4rodrig 20 0 13.4g 992828 257212 S 731.9 0.4 0:32.09 python3 103
 147254 p4rodrig 20 0 13.4g 991532 257216 S 731.2 0.4 0:33.23 python3 39
 147252 p4rodrig 20 0 13.6g 993024 256992 D 664.8 0.4 0:32.41 python3 2
 2722422 root 20 0 179880 46380 3300 S 3.3 0.0 92:48.99 pmdaproc 75
 147070 p4rodrig 20 0 60596 6156 3436 R 1.0 0.0 0:00.16 top 48
 2413 root 0 -20 0 0 0 I 0.3 0.0 0:00.24 kworker/23:1H-kblockd 23
 96335 root 20 0 0 0 0 I 0.3 0.0 0:00.01 kworker/80:3-events 80
 119798 root 20 0 0 0 0 I 0.3 0.0 0:00.01 kworker/10:2-events 10
 147392 root 0 -20 0 0 0 S 0.3 0.0 0:00.28 loop8 113
 3988028 root 20 0 0 0 0 I 0.3 0.0 0:01.09 kworker/108:0-events_power_efficie+ 108
 1 root 20 0 242528 11456 4480 S 0.0 0.0 17:12.39 systemd 105
 2 root 20 0 0 0 0 S 0.0 0.0 0:06.98 kthreadd 17
 3 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 rcu_gp 0
 4 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 rcu_par_gp 0
 5 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 slub_flushwq 0
 7 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/0:0H-events_highpri 0
 11 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 mm_percpu_wq 0
 12 root 20 0 0 0 0 S 0.0 0.0 0:00.00 rcu_tasks_rude_ 0
 13 root 20 0 0 0 0 S 0.0 0.0 0:00.00 rcu_tasks_trace 0
 14 root 20 0 0 0 0 S 0.0 0.0 0:04.31 ksoftirqd/0 0
 15 root 20 0 0 0 0 I 0.0 0.0 25:08.93 rcu_sched 64
 16 root rt 0 0 0 S 0.0 0.0 0:00.84 migration/0 0
 17 root rt 0 0 0 S 0.0 0.0 0:00.11 watchdog/0 0
```

- Stdout file will show
Pytorch Lightning
assigning ranks

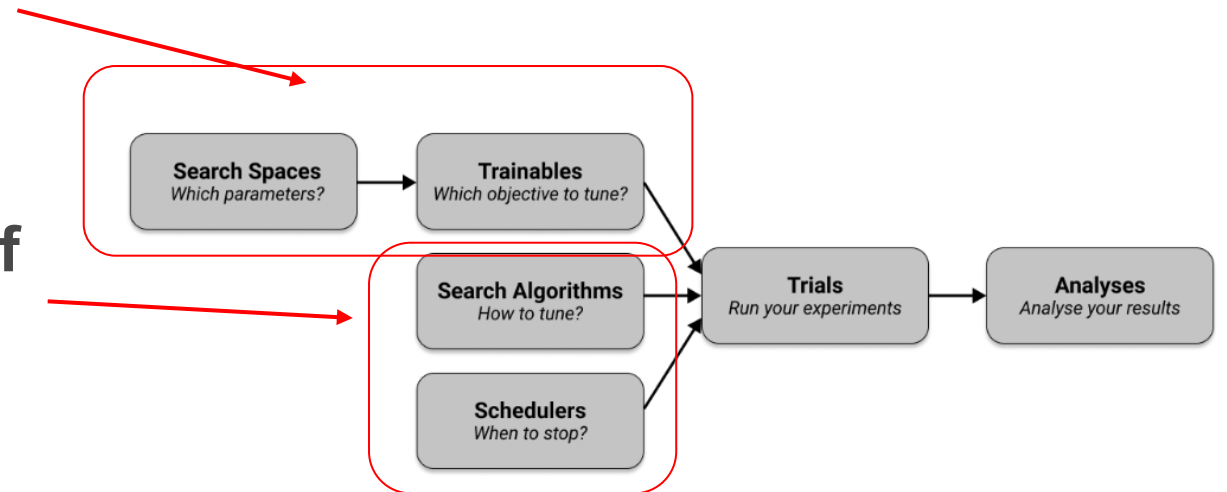
```
p4rodrig@exp-2-26:~/WKSHOPS/CIML26/ToolswScaling
py:75: Starting from v1.9.0, `tensorboardX` has been removed as a dependency of the `lightning.pyt
, due to potential conflicts with other packages in the ML ecosystem. For this reason, `logger=Tru
CSVLogger` as the default logger, unless the `tensorboard` or `tensorboardX` packages are found. F
stall lightning[extra]` or one of them to enable TensorBoard support by default
Initializing distributed: GLOBAL_RANK: 0, MEMBER: 1/8
Initializing distributed: GLOBAL_RANK: 1, MEMBER: 2/8
Initializing distributed: GLOBAL_RANK: 2, MEMBER: 3/8
Initializing distributed: GLOBAL_RANK: 7, MEMBER: 8/8
Initializing distributed: GLOBAL_RANK: 6, MEMBER: 7/8
Initializing distributed: GLOBAL_RANK: 3, MEMBER: 4/8
Initializing distributed: GLOBAL_RANK: 4, MEMBER: 5/8
Initializing distributed: GLOBAL_RANK: 5, MEMBER: 6/8
-----
distributed_backend=gloo
All distributed processes registered. Starting with 8 processes
-----
| Name | Type | Params |
-----
```

- **pause**

Ray tune

<https://docs.ray.io/en/latest/tune/key-concepts.html>

- Ray tune will set up hyperparameters and distributed workers
- Ray tune will manage execution of model for hyperparameter search

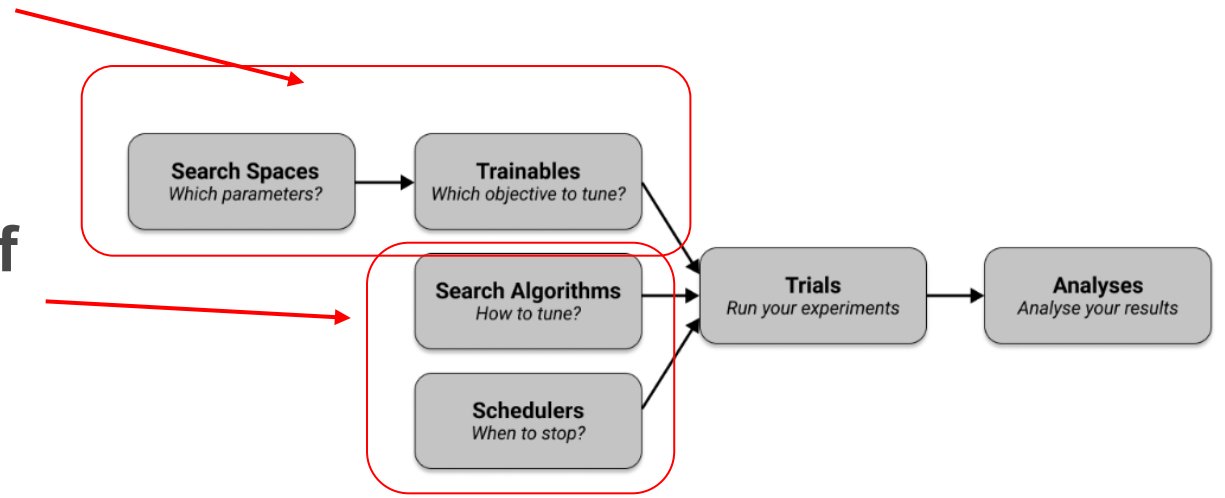


Ray Tune Trainables

Ray tune

<https://docs.ray.io/en/latest/tune/key-concepts.html>

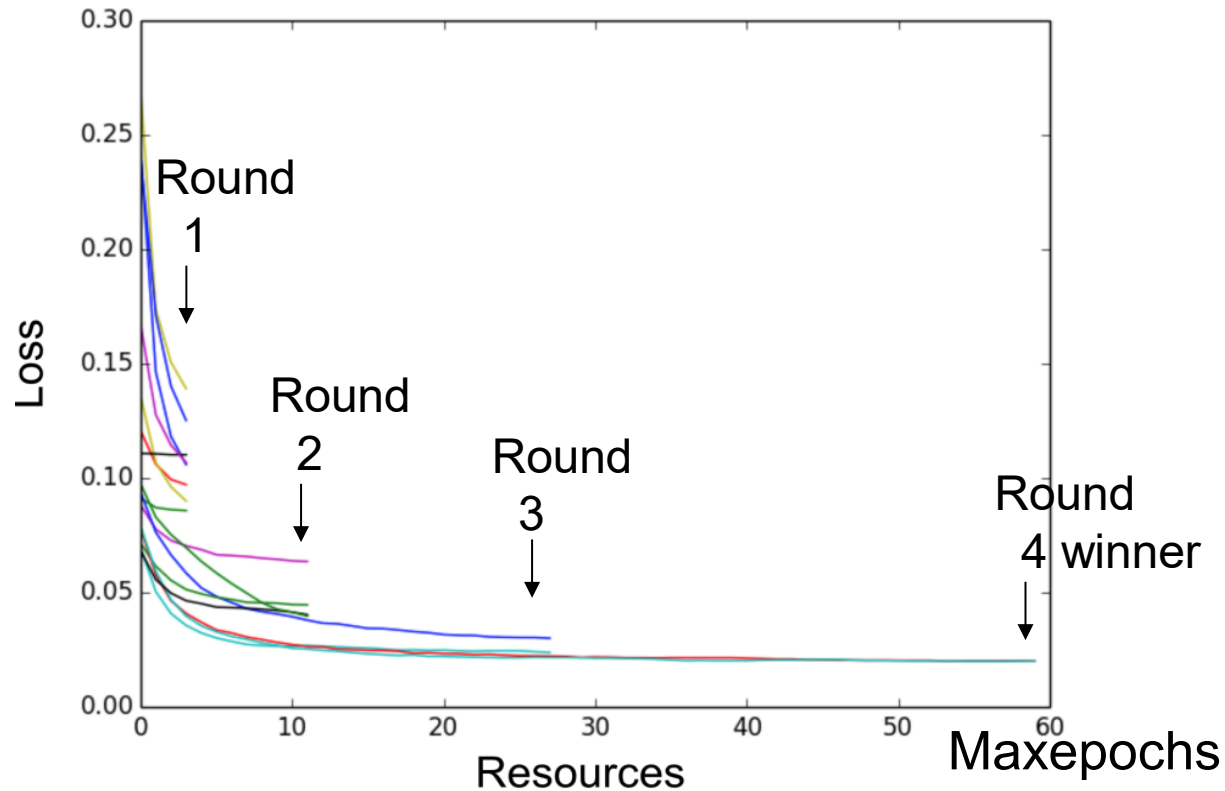
- Ray tune will set up hyperparameters and distributed workers
- Ray tune will manage execution of model for hyperparameter search
- You have to set up a training function with a 'config' argument
- Ray tune will stop workers according to metrics criteria



Ray Tune Trainables

A 'scheduler' example: Hyperband

Several brackets are run to end up with several possible overall winners.



Ray Tune framework

- You code the training loop, but Ray tune will monitor metrics and interrupt execution

```
def train_mnist(config):  
    get device, optimizer, data loaders,  
  
    while True:  
        run train step, test step, get loss  
  
    tune.report(metrics)  #Ray tune function here
```

Ray Tune framework

- The `tune.tuner` function has:

**metrics,
trial level scheduling/control,
parallel number to run**

```
mytune_config=tune.TuneConfig(
    metric    ="mean_accuracy",
    mode      ="max",
    scheduler = ASHAScheduler(...)
    num_samples=7, #num hypparm samples
    max_concurrent_trials=7 )

resources_p_trial = {"cpu": 1, "gpu": num_gpu}
```


Ray Tune framework

- The `tune.tuner` function has:

metrics,
trial level scheduling/control,
parallel number to run

resources,
hyperparameter ranges,

```
mytune_config=tune.TuneConfig(  
    metric    ="mean_accuracy",  
    mode      ="max",  
    scheduler = ASHAScheduler(...)  
    num_samples=7, #num hypparm samples  
    max_concurrent_trials=7 )  
  
resources_p_trial = {"cpu": 1, "gpu": num_gpu}  
  
tuner = tune.Tuner(  
    tune.with_resources(train_mnist,resources_p_trial),  
    tune_config=mytune_config, ...  
    param_space={  
        "lr": tune.loguniform(1e-4, 1e-2),  
        "numfilt": tune.choice([4, 8, 16]) },  
    )  
results = tuner.fit()
```

Ray Tune framework

- The `tune.fit()` execution sets up trials:

```
p4rodrig@login02:~/WKSHOPS/CIML26/RAY
Current best trial: f20a0_00001 with mean_accuracy=0.9762 and params={'lr': 0.0009214562210028149, 'numfilt
6}

Trial name      status      lr          numfilt     acc      iter    total time (s)
train_mnist_f20a0_00000  RUNNING    0.0057784      16    0.973      1      16.7734
train_mnist_f20a0_00001  RUNNING    0.000921456    16    0.9762     1      16.6879
train_mnist_f20a0_00002  RUNNING    0.000193831     4    0.9177     1      15.5841
train_mnist_f20a0_00003  RUNNING    0.00119102      4    0.9326     1      15.5848
train_mnist_f20a0_00004  RUNNING    0.000101152    16    0.9329     1      16.9812
train_mnist_f20a0_00005  RUNNING    0.00193723     16    0.9754     1      17.1681
train_mnist_f20a0_00006  RUNNING    0.00249482      8    0.9735     1      15.5762
```

- The Ray tune workers get launched:

```
p4rodrig@exp-3-12:~
top - 20:38:28 up 21 days, 4:26, 1 user, load average: 2.60, 0.69, 0.53
Threads: 6204 total, 10 running, 6173 sleeping, 0 stopped, 21 zombie
%Cpu(s): 0.1 us, 0.1 sy, 5.5 ni, 94.2 id, 0.1 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 257485.8 total, 231652.4 free, 20629.5 used, 5203.9 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 234948.7 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
3375197 p4rodrig 35   15   79.5g 517612 102072 R   99.7   0.2   0:15.24 ray::ImplicitFu
3375195 p4rodrig 35   15   79.5g 512288 101916 R   99.7   0.2   0:15.32 ray::ImplicitFu
3375181 p4rodrig 35   15   79.5g 512560 102008 R   99.7   0.2   0:15.45 ray::ImplicitFu
3375193 p4rodrig 35   15   79.5g 517392 102056 R   99.7   0.2   0:15.37 ray::ImplicitFu
3375201 p4rodrig 35   15   79.5g 524584 102020 R   99.7   0.2   0:15.11 ray::ImplicitFu
3375199 p4rodrig 35   15   79.5g 512780 102124 R   99.4   0.2   0:15.18 ray::ImplicitFu
3375177 p4rodrig 35   15   79.5g 515772 102000 R   99.0   0.2   0:15.62 ray::ImplicitFu
    13245 root      20    0   317324 121400  4232 S    3.9   0.0 294:04.81 perl
3370459 p4rodrig 20    0 3599040 147360  30640 R    3.2   0.1   0:00.84 ray::DashboardA
```

Ray Tune set up and running

I set up a conda environment for Ray

For single node execution:

- I used a `ray.init()` statement in the code
- Then I just ran `python mnist_raytune_v6.py`

For multiple node execution you might need to launch workers manually
(see <https://docs.ray.io/en/latest/cluster/vms/user-guides/community/slurm.html>)

Note: this command can help get ip addresses of your nodes
`mpirun -n Num_nodes -npernode 1 hostname -i`

- **pause**

Scaling in Deep Learning

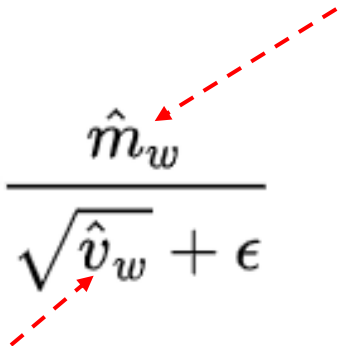
- **Two Goals**

1 Speed Up Learning - parallelizing data and/or model

**2 Optimize Memory - as models scale up they take up too much memory
e.g. V100s have 32Gb limit and 8B float32 parameters would fill that**

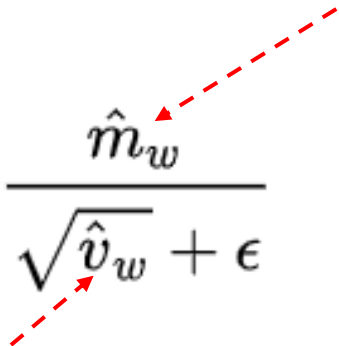
Deepspeed Memory Optimization

- Optimizers like Adam use a lot of memory **b/c** it tracks momentum and variances of gradients for *each* weight parameter update:

$$w^{(t+1)} \leftarrow w^{(t)} - \eta \frac{\hat{m}_w}{\sqrt{\hat{v}_w} + \epsilon}$$


Deepspeed Memory Optimization

- Optimizers like Adam use a lot of memory **b/c it tracks momentum and variances of gradients for each weight parameter update:**

$$w^{(t+1)} \leftarrow w^{(t)} - \eta \frac{\hat{m}_w}{\sqrt{\hat{v}_w} + \epsilon}$$


- “Zero Redundancy” (ZeRO) optimizes memory storage by partitioning these terms to different devices and then gathering them only when needed

ZeRO: Memory Optimizations Toward Training Trillion Parameter Models 2020, Rajbhandari et al, Microsoft

Deepspeed: 3 stages of incrementally more partitioning

1. Optimizer state partitioning (ZeRO stage 1)
2. Gradient partitioning (ZeRO stage 2)
3. Parameter (weights) partitioning (ZeRO stage 3)

Deepspeed: 3 stages of incrementally more partitioning

1. Optimizer state partitioning (ZeRO stage 1)
2. Gradient partitioning (ZeRO stage 2)
3. Parameter (weights) partitioning (ZeRO stage 3)

All options go in a json file and passed as argument

--deepspeed_config
ds_config.json

```
1-20:mnist_trialspipe$ more ds_config.json

"train_batch_size":16,
"bf16": { "enabled": true },
"fp16": { "enabled": false},
"gradient_clipping": 1.0,
"zero_optimization": { "stage": 0 },
"zero_allow_untested_optimizer": true
```

Deepspeed code snippets

Deepspeed initialization creates a “**model_engine**” to wrap the model

```
model_engine, opt, _, _ = deepspeed.initialize(model=model,  
model_parameters=model_params, args=args)
```

Deepspeed code snippets

Deepspeed initialization creates a “**model_engine**” to wrap the model

```
model_engine, opt, _, _ = deepspeed.initialize(model=model,  
model_parameters=model_params, args=args)
```

The training loop now uses **model_engine** for forward,backward processing

```
output = model_engine(data)  
loss = loss_function(output, target)  
model_engine.backward()  
model_engine.step()  
htcore.mark_step()
```

Launch program instances with mpirun, or deepspeed launcher

A quick Deepspeed test, using mnist with extra layers (1.356B parameters)

Stage	Python max memory usage (train loop memory usage)	Avg samples per sec
1	~27.1GB (24.6GB)	~79
2	~10.3GB (7.9 GB)	~88
3	~13.3GB (5.0 GB)	~73

Note: the were short tests with small data, using 1 HPU node (8 devices). Also, other parameters will impact memory, like model precision

`torch.cuda.mem_get_info()` is a function that help get this info

- End